

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad I: Desarrollo Web Frontend

Tema: HTML5, XML, CSS3, Flexbox/Grid, Diseño Responsivo y SEO

Modalidad: Presencial / Laboratorio

Duración: 90 minutos

APELLIDOS Y NOMBRES:

- Alarcón Mendoza Estiven Rodrigo
 - Calderon Leiva Anna Olenka
 - Cruz Cruz Alexander Jhon
 - Martinez Casas Cristhian Emilio
-

Link del proyecto: https://github.com/AnnaOlenka/practica_grupo1_s05

Paso 1: Crear el proyecto con Vite

```
npm create vite@latest panel-metricas -- --template react cd panel-metricas
npm install
npm install prop-types
npm run dev
```

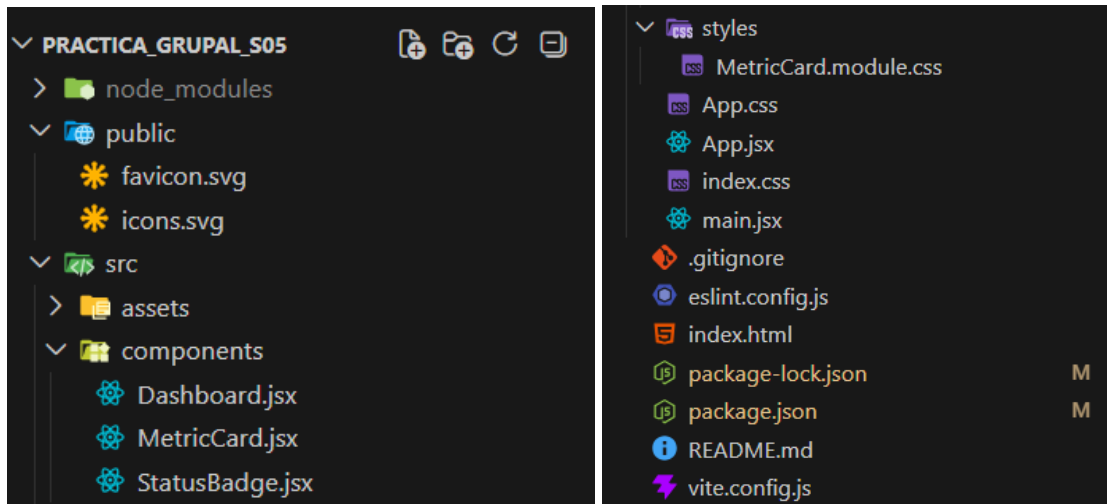
```
> panel-metricas@0.0.0 dev
> vite

VITE v8.0.11 ready in 199 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Paso 2: Estructura del proyecto

La estructura de carpetas fue definida por el Arquitecto de Proyecto usando Vite como bundler.



Paso 2.1: vite.config.js

Se configuraron alias de importación para evitar rutas relativas largas (`../..`).

`@components` apunta a `src/components/` y `@styles` a `src/styles/`.

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import path from 'path'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'),
      '@components': path.resolve(__dirname, './src/components'),
      '@styles': path.resolve(__dirname, './src/styles'),
    },
  },
})
```

Paso 3: Desarrollo del código

Paso 3.1: main.jsx

Desde este archivo se monta el componente raíz <App/> dentro del elemento #root del index.html. A partir de ahí, React comienza a renderizar toda la aplicación.

```
main.jsx X
practica_grupal_s05 > src > main.jsx
1 import { StrictMode } from 'react'
2 import { createRoot } from 'react-dom/client'
3 import './index.css'
4 import App from './App.jsx'
5
6 // CSR: React hidrata el DOM aquí. El HTML inicial solo tiene <div id="root">
7 createRoot(document.getElementById('root')).render(
8   <StrictMode>
9     <App />
10  </StrictMode>,
11 )
```

```
main.jsx
-> App.jsx
  -> Dashboard.jsx
    -> MetricCard.jsx
      -> StatusBadge.jsx
```

Paso 3.2: App.jsx

```
App.jsx X
practica_grupal_s05 > src > App.jsx > ...
15 const METRICS_DATA = [
70 ]
71
72 /**
73  * Componente raíz – orquesta props hacia Dashboard.
74  * Arquitectura CSR: este componente se hidrata en runtime, no en el
75  */
76 function App() {
77   return (
78     <Dashboard
79       title="Panel de Métricas Académicas"
80       subtitle="Universidad Nacional del Centro del Perú – IS093A"
81       metrics={METRICS_DATA}
82     />
83   )
84 }
85
86 export default App
```

En App.jsx se definen los datos estructurados de las métricas y se pasan hacia Dashboard mediante props, Esto demuestra el flujo unidireccional de datos, porque la información viaja desde el componente padre hacia los componentes hijos:

```
App -> Dashboard -> MetricCard
```

En Dashboard.jsx, las métricas recibidas se recorren con `.map()` y cada objeto se envía a MetricCard como props:

```

Dashboard.jsx X
practica_grupo1_s05 > src > components > Dashboard.jsx > ...
14 function Dashboard({ title, subtitle, metrics }) {
15   </header>
21
22   /* Grid responsivo via CSS inline – el Ingeniero de Estilos puede moverlo a módulo */
23   <div
24     style={{
25       display: 'grid',
26       gridTemplateColumns: 'repeat(auto-fill, minmax(280px, 1fr))',
27       gap: '1.25rem',
28     }}
29   >
30     {metrics.map((metric) => (
31       <MetricCard
32         key={metric.id}
33         title={metric.title}
34         value={metric.value}
35         unit={metric.unit}
36         progress={metric.progress}
37         status={metric.status}
38         description={metric.description}
39       >
40         /* children: composición – el badge de estado también se pasa como hijo */
41         <StatusBadge status={metric.status} />
42       </MetricCard>
43     ))}
44   </div>
45 </main>
46 )
47 }
48

```

Aquí se aplican dos conceptos importantes:

Props:

Se usan para enviar datos como title, value, unit, progress, status y description desde Dashboard hacia MetricCard.

Children:

Se usa para insertar contenido adicional dentro de un componente. En este caso, StatusBadge se pasa como contenido hijo de MetricCard:

```

<MetricCard
  key={metric.id}
  title={metric.title}
  value={metric.value}
  unit={metric.unit}
  progress={metric.progress}
  status={metric.status}
  description={metric.description}
>
  /* children: composición – el badge de estado también se pasa como hijo */
  <StatusBadge status={metric.status} />
</MetricCard>

```

Paso 3.3: MetricCard.jsx

Luego, en MetricCard.jsx, ese contenido se recibe con children:

```
MetricCard.jsx X
practica_grupal_s05 > src > components > MetricCard.jsx > ...
25 function MetricCard({ title, value, unit, progress, status, description, children }) {
26   const progressColor = PROGRESS_COLORS[status] ?? PROGRESS_COLORS.info
27 }
```

Se documentan los props esperados:

```
/**
 * Tarjeta individual de métrica.
 * Recibe datos vía props y acepta children para composición visual adicional.
 *
 * @param {{
 *   title: string,
 *   value: string|number,
 *   unit: string,
 *   progress: number,
 *   status: import('./StatusBadge').BadgeStatus,
 *   description: string,
 *   children?: import('react').ReactNode
 * }} props
 */
```

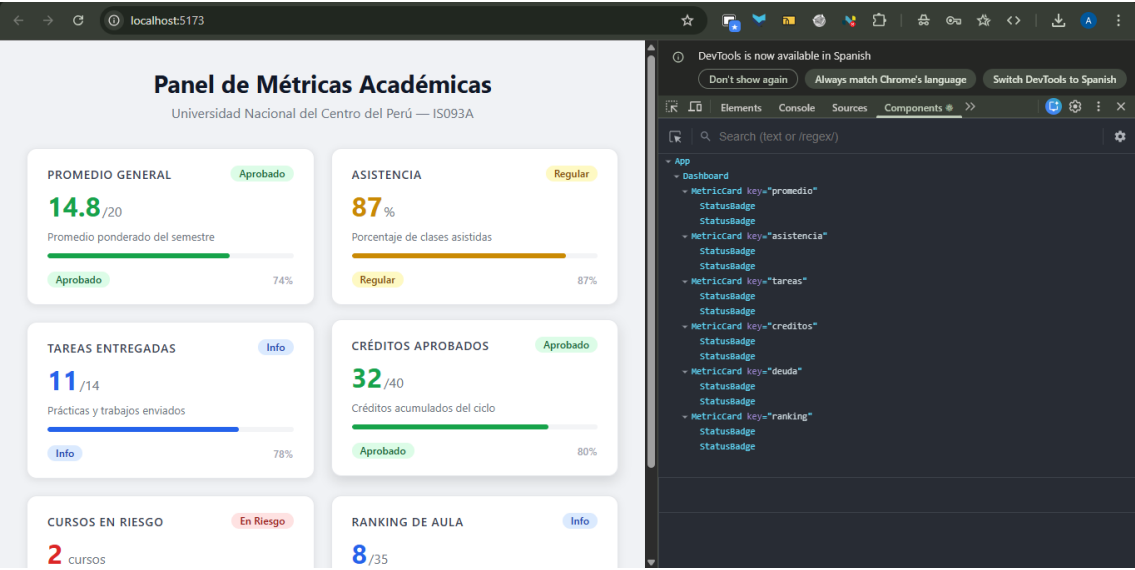
Paso 3.4. Estilos

El archivo src/styles/MetricCard.module.css contiene los estilos específicos del componente MetricCard.

Se usa CSS Modules, lo que significa que las clases definidas en este archivo no son globales. React/Vite transforma los nombres de clase en identificadores únicos para evitar colisiones con otros componentes.

```
# MetricCard.module.css X
practica_grupal_s05 > src > styles > # MetricCard.module.css > ...
1  /* CSS Module: los selectores se convierten en hashes únicos (_card_1a2b3c)
2     para evitar colisiones con estilos globales. */
3
4  .card {
5     background: #ffffff;
6     border-radius: 12px;
7     padding: 1.25rem 1.5rem;
8     box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
9     border: 1px solid #e5e7eb;
10    display: flex;
11    flex-direction: column;
12    gap: 0.75rem;
13    transition: transform 0.2s ease, box-shadow 0.2s ease;
14  }
15
16  .card:hover {
17    transform: translateY(-3px);
18    box-shadow: 0 6px 16px rgba(0, 0, 0, 0.12);
19  }
20
21  .header {
22    display: flex;
23    justify-content: space-between;
24    align-items: flex-start;
25    gap: 0.5rem;
26  }
27
```

Paso 4: React DevTools



Árbol:

```

  Árbol

  <App>
    <Dashboard>
      <MetricCard key="promedio">
        <StatusBadge>
        <StatusBadge>
      <MetricCard key="asistencia">
        <StatusBadge>
        <StatusBadge>
      <MetricCard key="tareas">
        <StatusBadge>
        <StatusBadge>
      <MetricCard key="creditos">
        <StatusBadge>
        <StatusBadge>
      <MetricCard key="deuda">
        <StatusBadge>
        <StatusBadge>
      <MetricCard key="ranking">
        <StatusBadge>
        <StatusBadge>
    </Dashboard>
  </App>

```

Props:

```
Dashboard props
{
  "title": "Panel de Métricas Académicas",
  "subtitle": "Universidad Nacional del Centro del Perú – ISO93A",
  "metrics": [
    "{description: \"Promedio ponderado del semestre\", id:...}",
    "{description: \"Porcentaje de clases asistidas\", id:...}",
    "{description: \"Prácticas y trabajos enviados\", id: ...}",
    "{description: \"Créditos acumulados del ciclo\", id: ...}",
    "{description: \"Cursos con asistencia crítica\", id: ...}",
    "{description: \"Posición en el grupo académico\", id:...}"
  ]
}
```

```
promedio >> MetricCard props
{
  "title": "Promedio General",
  "value": 14.8,
  "unit": "/20",
  "progress": 74,
  "status": "success",
  "description": "Promedio ponderado del semestre",
  "children": "<StatusBadge />"
}
```

Resultado Final:

